Bal's Sound Garden: Procedural World, Music Synthesis and Story Simulation

System Architecture and Implementation Report

ABProjects

October 2026

Abstract

Bal's Sound Garden is a desktop music-making adventure implemented in Python with Pygame and NumPy. A cat explores a continuously generated 2.5D island, collects authored or procedurally synthesized sounds, builds arrangements in a multitrack editor and serves animal clients in a time-dependent music shop. The implementation couples visual world generation, deterministic component identities, a lightweight composition model, waveform synthesis, scheduled playback, request evaluation, and durable user data. This report explains those subsystems from the shipped source, including the deterministic boundaries, the data path between exploration and exporting, quantitative scoring rules, and failure-handling measures. The examined source contains 90 application Python modules and 300 asset files; 23 supplied regression scripts cover focused application and release behavior. Runtime modules and assets were preserved byte for byte while the public repository was organized. Syntax and packaging structure were checked for this preparation, but interactive gameplay tests were not rerun in the current environment. The report makes no unmeasured frame-rate, audio-latency or quality claims.

Introduction

Bal's Sound Garden combines an exploration game and a small digital music workstation. The player is Bal, a cat collecting usable musical fragments in a painted island landscape. A sound discovery is not simply a score token; it refers to a component whose audio can be previewed, planted into the world, arranged in a project, played through a gramophone, or delivered as part of a shop request. This coupling creates the principal engineering problem: one component identity must survive several different uses without changing meaning.

A Story profile introduces an ongoing client-facing shop. Customers arrive, state musical requirements, wait, leave, return for scheduled work, and reward compositions that satisfy their briefs. An Explore profile retains free-form collection and composition. These modes share much of the world and sound machinery but use distinct progress data. In both, the Music Maker is a functional editor rather than a separate simulation: the arrangement it saves is the one used by playback and WAV export.

The design is intentionally a desktop application rather than a browser executable. Pygame supplies the window, event system and software-surface drawing interface. NumPy provides waveform arrays and vector operations. The application includes third-party runtime dependencies and approximately 300 bundled images and effects. The accompanying GitHub Pages website explains the system; it does not execute the game.

System architecture

The entry point `main.py` delegates to `cat_society.app:main`. The application layer creates the game window, input services, audio engine and active screen. `cat_society/screens` contains the main menu, exploration, Story, Music Maker, Songbook and project export interfaces. `cat_society/world`, `render`, `entities` and `systems` implement world state, traversal, painting and animal behavior. `cat_society/components`, `audio` and `music` manage sound components and projects. `cat_society/story` manages clients, deadlines, shop operations and fulfillment.

This is an event-driven program with multiple timescales. Screen event handling responds to player actions. The world display advances at frame cadence. Audio buffers are generated at a sample rate. The Story clock advances in game minutes as real seconds pass, even while the composition screen is active. Keeping those concepts distinct prevents the visual frame rate from becoming the sole determinant of musical duration or a customer's contractual deadline.

The source boundaries are: `app.py` and `screens/` for event handling; `world/`, `entities/` and `systems/` for world state; `projection.py`, `camera.py` and `render/` for the painted scene; `components/` for the authored and generated sound catalogue; `music/` and `audio/` for composition and playback; `story/` for clients and scoring; and `persistence.py`, `paths.py` and `music/store.py` for player data.

The application contains no cloud runtime dependency for its main game loop; local player files, audio and assets are handled on the machine. This does not imply that all possible operating systems or audio devices have been fully regression-tested.

Procedural landscape and projection

Stable world chunks

The open island uses chunk-addressed generation. `World.scenery(gx, gz)` caches content by grid coordinates. A local pseudorandom generator is initialized from a seed hash of the chunk and a fixed stream discriminator. This yields stable procedural placement for a chosen world seed, so revisiting the same area reproduces its ground objects rather than drawing new random values on each frame.

The code deliberately separates random streams for gameplay scenery, decorative scenery and local ground cover. The generation functions initialize generators with distinct stream constants such as `0x5CE`, `0xDEC0` and `0x6A55`. This separation matters because adding a decorative flower should not silently reshuffle trees, bushes and other interactable content. Water placement is governed independently; for example, `land_allowed` excludes trees and bushes from nonzero water

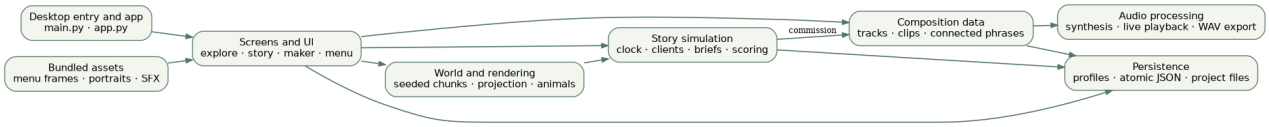


Figure 1: Major runtime subsystems and their source boundaries. This is a conceptual dependency/data-flow diagram, not a runtime call graph.

depth and uses depth thresholds for smaller objects and shoreline stones.

For a fixed seed s, grid coordinate (g_x, g_z), and stream discriminator d, a generated cell may be expressed as

$$C(g_x, g_z, d) = G(\mathrm{PRNG}(H(s, g_x, g_z, d))),$$

where H is the project's seed-mixing procedure and G the placement rule for that stream. This is a description of the source's determinism boundary, not a cryptographic guarantee. Reproducibility presumes the same implementation version and the same state/configuration.

Rendering model

World-space positions are transformed into a perspective-influenced two-dimensional presentation by `projection.py` and `camera.py`. Render components draw water, ground, trees, animals, sprouts, props, atmosphere and day/night layers as Pygame surfaces. Independent render modules allow the game to compose an illustrated scene without a conventional external 3D engine. The camera and rendering passes must remain consistent about depth ordering and scale: a world object that is correctly placed in simulation still needs a corresponding projected hit region, shadow and occlusion relationship on screen.

The atmosphere system changes the palette across dawn, daytime, sunset, dusk and night. The source includes seeded auroras, night light from sound objects, and animation frames for the main menu. The menu has 256 JPG frames; these are bundled assets, not compiled platform installers. Separating cached background work from live interaction prevents the renderer from repeating some expensive operations, although the current report does not attribute a measured frame-time improvement to specific caches.

Sound catalogue and component identity

The sound library combines a finite curated catalogue (`components/catalog.py` and `curated_library.py`) with a procedural generator (`components/procedural.py`). The curated set provides predictable notes, chords, instruments, rhythms, ambience and effects. Procedural items extend that set without prebuilding every possible waveform or embedding a separate audio file for each variant.

ComponentDNA generation takes a seed and constructs a structured sound specification: tonal center and scale, voice selection, oscillator characteristics, envelope and filtering, optional layers, motif or chord structure, rhythmic placement, and effects. The result is musical data that the renderer can synthesize later. Some components describe a single event; others encode phrases, rhythms or evolving ambient layers. Rarity influences generated complexity rather than only changing a collectible color.

Crucially, the generator preserves identifier formats across versions. New discoveries are represented using identifiers of the form `proc2.<seed>`, while the implementation retains decoding for earlier `proc.<seed>` components. A saved Music Maker project can therefore preserve its sound identities without storing huge rendered audio buffers in each project file. Compatibility still depends on retaining the appropriate reconstruction algorithms; identifiers alone do not guarantee sound equality if the generator implementation is changed later.

There is a practical division between content selection and content realization: the collection and sequencer store component identities; `audio/synth.py` resolves those identities into sampled sound. The catalogue and procedural generator should not be confused with a machine-learning system. The inspected implementation is rule/seed based and does not claim trained generative models.

Composition and waveform processing

Arrangement data model

The music project model is defined in `music/project.py`. A `Project` has global fields such as tempo and bar count, while each `Track` carries a family, clips, mute/volume/effect state and optional connected phrases. A `Clip` stores a component identifier and a step index. The duration of a clip comes from the referenced component rather than an arbitrary stretch applied to every item.

The new-project defaults are 100 BPM and eight bars. The underlying grid has four beats per bar and two eighth-note steps per beat, producing eight steps per bar. At tempo B beats/minute, the duration of one eighth-note step is

$$\Delta t_{\mathrm{step}} = \frac{60}{2B} \quad \text{seconds.}$$

At the default 100 BPM this is 0.3 seconds; an eight-bar project spans 64 grid steps or 19.2 seconds before effects and note-release tails. This timing formula describes the nominal grid; the audible waveform may extend past a tile due to its synthesis envelope or connected phrase behavior.

The project layer prevents exact duplicate clip placements on the same step, validates loaded project fields and repairs connected phrases when referenced clips have been removed. That last operation (`Track.heal_phrases`) is important: a deletion in the UI must not leave phrase metadata pointing to nonexistent clips. Family-specific rules prevent, for example, placing a drum component on a melody-only track.

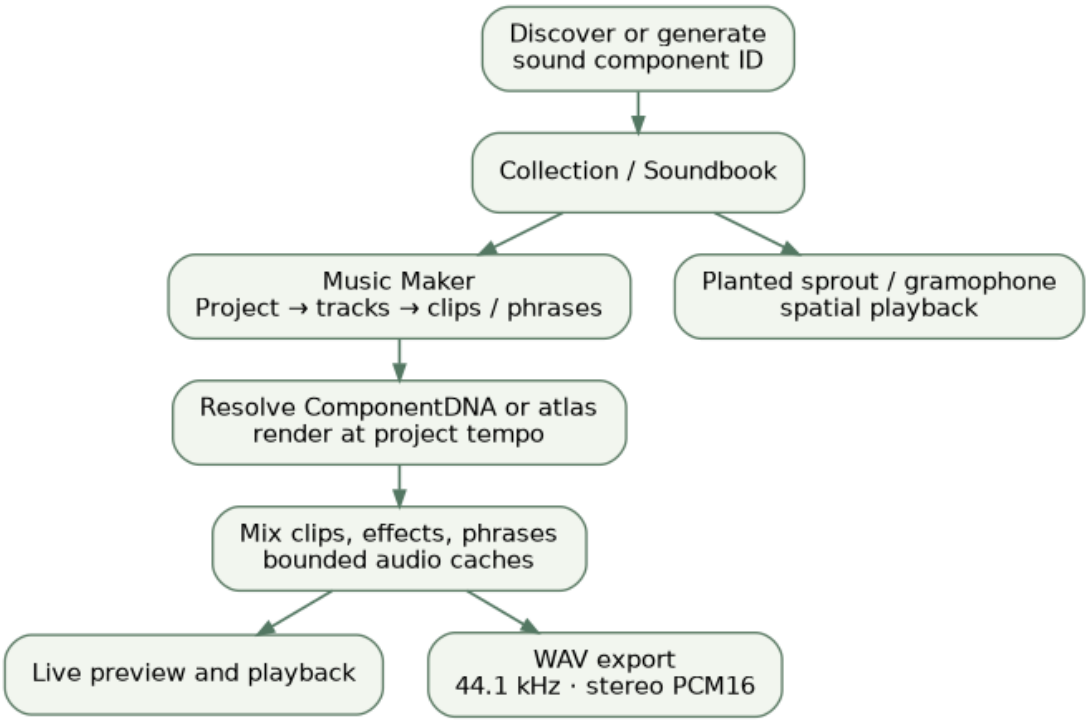


Figure 2: Component identities connect discovery, composition, synthesis, live playback and WAV output.

Synthesis and playback

`audio/synth.py` synthesizes waveforms at $f_s = 44{,}100$ samples per second. Sound construction includes oscillators, ADSR envelopes, frequency shaping, noise and layered voice components. Musical time is converted into sample coordinates for playback and export. If a nominal signal duration is T, the buffer requires approximately

$$N = \lceil f_s T \rceil$$

samples, with additional tail handling where a sound envelope or effect requires it. The audio pipeline is implemented in floating-point arrays before its final playback or PCM formatting stages.

The Music Maker uses the same project representation for live playback, world gramophones and file export. It supports ordinary single clips as well as connected phrases. Sustained notes can be rendered as a shared attack/body/release; chords can maintain voice continuity, while discrete percussion stays as separate hits. These are materially different musical behaviors, so the renderer resolves connections by component family rather than treating all connected tracks as one generic long sample.

Bounded intermediate work

`audio/export.py` renders repeated components and their effects through bounded `OrderedDict` caches. The component cache limit is 64 entries and the effect cache limit is 32 entries in the inspected code. The intended property is a bounded number of cached sound objects rather than a permanent memo for every seed, component and edited effect. Memory use is still affected by the size of individual buffers, active mix windows and any simultaneous sound-playing objects, so cache-entry limits alone are not a global RAM ceiling.

Completed projects can be exported to conventional stereo WAV. `export_wav` writes 44.1 kHz, 16-bit PCM with two channels, rather than introducing a proprietary export format. The waveform is generated before the final destination file is replaced; the resulting playback/export consistency is an engineering goal supported by a shared render path, but exact sample-for-sample parity across live playback devices has not been independently measured here.

Story simulation and musical evaluation

Game clock and client lifecycle

The Story clock (`story/clock.py`) stores a monotonic day-relative stamp and separately computes human-readable clock time. A complete game day lasts twenty real-time minutes under the inspected configuration, and the shop is open from 09:00 to 18:00 in game time. The advancement factor is

$$r = \frac{1440 \text{ game minutes}}{20 \cdot 60 \text{ real seconds}} = 1.2 \ \frac{\text{game minutes}}{\text{real second}}.$$

The implementation emits boundary events when simulated time crosses opening, closing or day-change thresholds. A monotonic stamp makes appointment and deadline comparisons simpler than comparing formatted “day HH:MM” strings. The shop can continue advancing while Bal is composing, and stops when the overall game enters a paused condition.

Animal clients are simulated as actors with movement, queueing, waiting, dialogue and appointment states (`story/clients.py`). Their requests are structured rather

than pure conversational judgement. This allows musical results to be scored reproducibly and explained to the player. Shop upgrades, decorative objects, helper contracts, coins and reputation are modeled through separate Story services and state objects.

Explainable evaluation

`story/scoring.py` combines six scores in $[0, 1]$: required musical families, mood tags, tempo fit, energy, effort and cohesion. The raw score for a song is

$$S_0 = 0.25F + 0.30M + 0.15T + 0.10E + 0.10R + 0.10C - P,$$

where P penalizes an unwanted mood tag. The source caps the score to at most 0.35 if required musical families are missing. It then applies a freshness decay

$$S = \mathrm{clip}_{[0,1]}(\min(S_0, \mathrm{cap}) \cdot 0.60^k),$$

where k counts prior sales of the same arrangement (the decay constant is configuration-backed and defaults to 0.60). A commission passes at the configured threshold, 0.55 by default. The algorithm also produces named score parts and explanatory notes such as missing instrumentation, insufficient development or too many competing moods. This provides legibility to the player: shop outcomes depend on structured properties of the song rather than hidden subjective choices.

The calculation also incorporates content hashing (`project_content_hash`). Renaming a song is not intended to reset the novelty penalty because the arrangement, not merely its display name, is hashed. The scoring rule is a game design objective, not a musicological assessment of artistic merit or a listener-study result.

Persistent data and failure handling

A music game combines several kinds of state: collected sounds, Songbook projects, rendered WAV exports, world placement and Story/shop progression. Some are recomputable from a seed or component identifier, while others represent irreplaceable player decisions. The persistence layer should make that distinction visible.

`persistence.atomic_write_json` serializes a payload and immediately re-parses it in memory, writes to a temporary file inside the target directory, flushes the file, reopens and parses the temporary copy, then atomically replaces the destination through `os.replace`. When supported by the platform it synchronizes the directory. A failure removes the temporary file and raises a controlled `PersistenceError`; this is safer than writing JSON directly over an older save.

The export path uses a related but distinct transaction: it writes the WAV into a temporary sibling, completes/flushes it, then replaces the final WAV path atomically. The program keeps source/application installation files separate from writable player data in packaged environments. Malformed project data is validated on load so corrupt content can be rejected without silently mutating a valid file already on disk.

These measures protect against many interrupted-write cases, but atomic replacement is not a complete backup system. Disks can fail, devices can lose both old and new data, and file systems differ in durability guarantees. The source does not claim to provide cross-device transactional behavior or cloud recovery.

Performance design and operational boundaries

The runtime employs several performance-oriented boundaries rather than a single optimization switch. World chunks and scenery results are cached by coordinates. Decorative and gameplay random streams are separated to preserve stable generation. Audio synthesis renders reusable components into buffers and caches a bounded subset, while project mixing avoids retaining unlimited intermediate results. Screen transitions and Pygame drawing control presentation work in the foreground process.

These techniques address predictable classes of overhead: repeated procedural work, repeated audio rendering, and uncontrolled buffer retention. Their practical benefit depends on player action, component duration, device speed, audio drivers and the operating system. The project provides no benchmark in the supplied archive establishing an exact FPS improvement or a comparative render-time advantage over another implementation. Accordingly, this report does not invent performance percentages.

There are also scale trade-offs. A catalogue with many authored components increases discoverable variety but also demands reliable indexing and search in the Soundbook. Dense projects may require long mix buffers. Story-client simulations introduce evolving state that must not run through a pause. Generated world content is locally stable for a seed and version but is not necessarily reproducible across incompatible code versions. The renderer's 2.5D approach provides a distinctive painted look while retaining projection and draw-order complexity.

Verification and reproducibility

The supplied build-source archive includes 23 named regression scripts and two packaging tools, alongside source metadata and a PyInstaller specification. Test topics include application integration, procedural reconstruction, sound matching, Music Maker input behavior, Story/shop actions, save durability, export portability, and release packaging. These scripts offer concrete places to verify changes; the existence of a script is not the same as a successful execution in the current environment.

For this repository preparation, all executable package code, `main.py`, bundled assets and the PyInstaller specification were copied and checked byte-for-byte against their originals: **392 original runtime/build files match**. The 23 regression scripts were moved into `tests/`; three scripts received only path-root adjustments necessitated by the directory move. Static Python compilation passes for the source, tools and tests. An executable smoke test of game screens, synthesis, and exported WAV was not performed in this environment because Pygame was not installed. Build instructions and test commands are supplied for an appropriate local desktop environment.

The verification matrix is deliberately limited. Application file integrity and Python syntax checks passed. The supplied integration, musical-generation, Story/shop, WAV export and packaging test scripts remain available but were **not executed** in this environment. Cross-platform PyInstaller packaging and

actual gameplay require suitable machines with Pygame and audio drivers. GitHub Pages is checked as a static document and does not run the Python game.

Distribution, licences and limitations

This repository is a **source and engineering presentation**. GitHub Pages serves an explanatory page originally authored for the developer's portfolio. Released game executables stay on itch.io; the repository need not publish GitHub Releases. Static images on the Pages site are presentation assets and are not a browser port of the Pygame desktop game.

The uploaded archive states that public licence selection and the provenance of original game media have not been fully confirmed. The project owner must approve reuse terms for the code and art; Pygame, NumPy, SciPy and other bundled or installed third-party material retain their own licences. A publicly visible repository by itself is not equivalent to an approved open-source grant. These ownership constraints are distribution work, not defects of the music or simulation algorithms.

Conclusion

Bal's Sound Garden uses a stable component identity to join world exploration, algorithmic synthesis, a track-based composition editor and a timed shop simulation. The architecture is significant because the same sound objects cross several domains: objects in the world, collection entries, audio buffers, clip placements, scored compositions and WAV exports. Deterministic generation, structured music projects, bounded audio caches, explicit Story time, explainable scoring and transactional file writes support that continuity. The examined repository preserves the shipped application code and focuses its public materials on how those systems work; further claims about platform reliability or speed require executable regression and real-hardware measurements.

Source references

1. ABProjects, `cat_society/app.py`, `screens/`, `world/`, `render/`, `entities/` and `systems/`, application and world implementation.
2. ABProjects, `components/catalog.py`, `components/curated_library.py` and `components/procedural.py`, component identities and synthesis specifications.
3. ABProjects, `audio/synth.py`, `audio/engine.py`, `audio/export.py`, waveform generation, playback and WAV publishing.
4. ABProjects, `music/project.py`, `music/store.py`, `screens/maker.py`, arrangement representation and music editing.
5. ABProjects, `story/clock.py`, `story/clients.py`, `story/scoring.py`, simulation time and commission evaluation.
6. ABProjects, `persistence.py`, `paths.py`, durable state and per-user file locations.
7. ABProjects, `tests/` and `tools/build_release.py`, available regression and packaging scripts.
8. ABProjects, Bal's Sound Garden on itch.io, game description and binaries.